

Programmazione 1

Corso c

25/09/2023
> Il linguaggio C

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

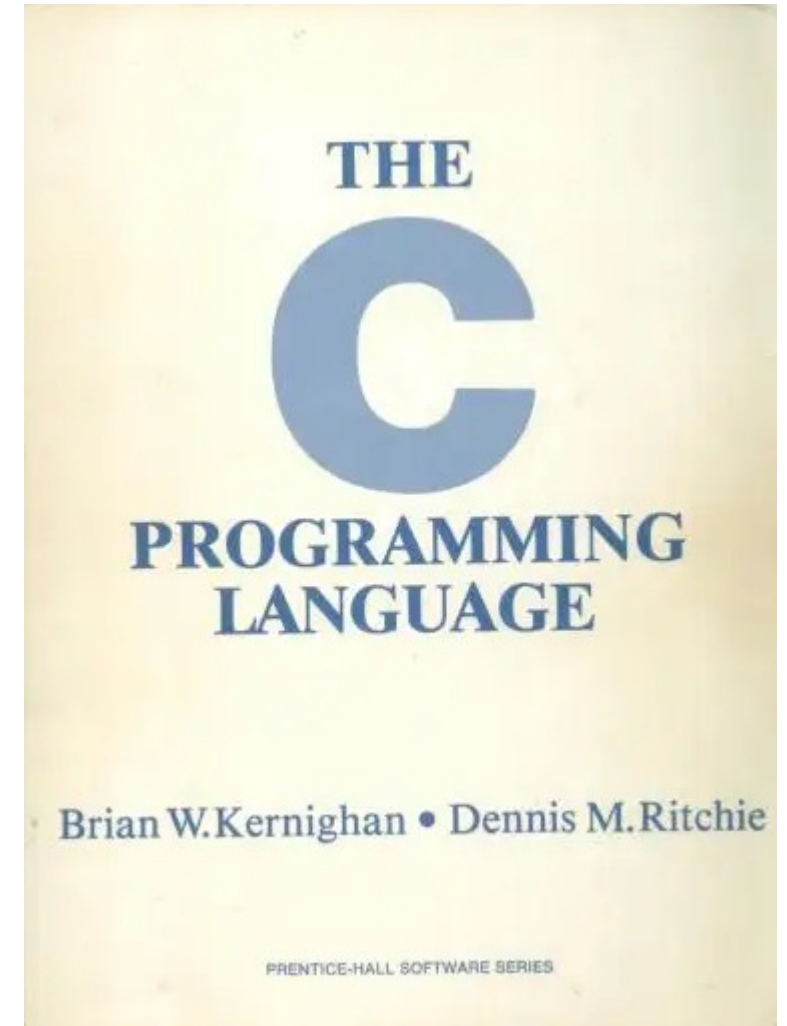
Il linguaggio C

- Nel 1969 Ken Thompson (Bell Labs, AT&T) inizia a scrivere un sistema operativo per il chiamato Unix in assembly PDP-7
- Ben presto ci si accorge che scrivere un S.O. in codice assembly è un lavoro troppo lungo
- Il collega Denis Ritchie sviluppa allora un linguaggio chiamato C per riscrivere Unix



Il linguaggio C

- Inizialmente sviluppato (K&R C) per uso interno sui computer PDP-7 e PDP-11
- Verso la fine degli anni '80 è standardizzato da ANSI e ISO per uso su una varietà di hardware differente (ANSI-C, C-89)
- Lo standard ha subito numerose numerosi aggiornamenti (C-99, C-11, etc.)
- Il libro di testo fa riferimento a C-11



Il linguaggio C

Il C è un linguaggio di programmazione

compilato,

un compilatore traduce i sorgenti C nel linguaggio macchina del caso

imperativo,

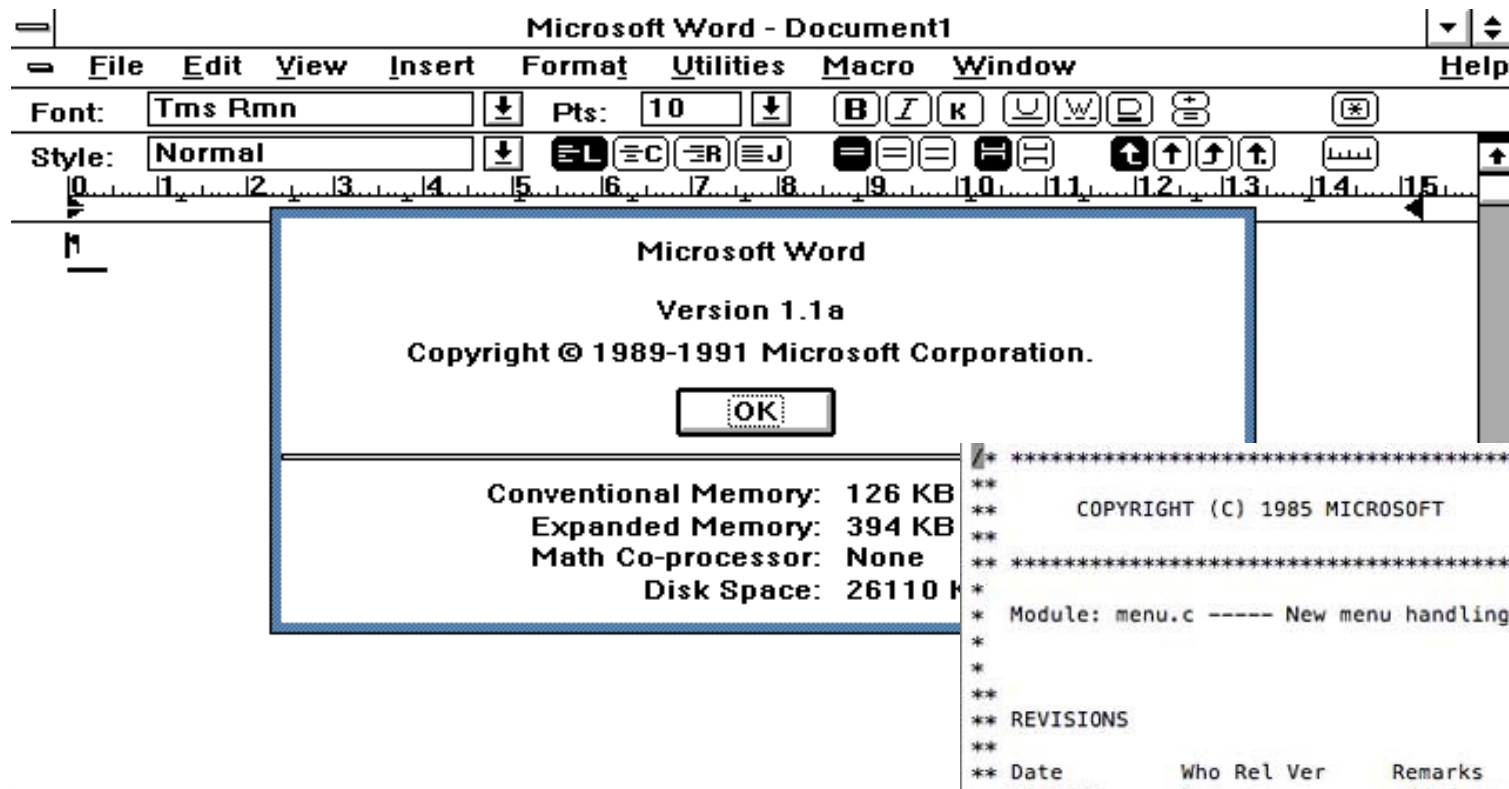
il programma è un insieme di istruzioni pensate come ordini per il compilatore

strutturato,

il codice sorgente è organizzato in blocchi racchiusi da delimitatori

fortemente tipizzato

il programmatore deve specificare il tipo di ogni variabile



For Help, press F1

Word 1.1a

Linguaggio C

Sorgente:

<http://www.computerhistory.org/atcm/microsoft-word-for-windows-1-1a-source-code/>

```

*****
**
**      COPYRIGHT (C) 1985 MICROSOFT
**
** *****
*
* Module: menu.c ----- New menu handling routine
*
*
**
** REVISIONS
**
** Date      Who Rel Ver    Remarks
** 11/8/85   bz              added status area view menu item
** 12/5/85   dsb             added renumber paragraphs menu item
** 2/14/86   dsb             added Zoom command for Page Preview
** 2/20/86   bz              Changed whole ballgame to allow CAB-style
**                               commands and allow menu customization
** 7/3/86    bz              Simplified the mnst initialization scheme
** 7/24/86   dsb             revamped to match new spec
** 10/28/86  bac             Redid everything
** 6/26/87   yxy             Undo strings (bac moved 'em to menu1.c)
**
** ***** */

#define NOGDICAPMASKS
#define NOSYSMETRICS
#define NOICON
#define NOKEYSTATE
#define NORASTEROPS

```




Doom 3

Linguaggio C

Sorgente:

<http://fabiansanglard.net/doom3/index.php>

```
#ifndef _MAIN_H_
#define _MAIN_H_

#include "idlib/precompiled.h"

#include "../doom/doomlib.h"
#include "../doom/doominterface.h"
#include "../doom/globaldata.h"

// DHM - Nerve :: Enable demo recording for game clips
#define _DEMO_RECORDING

#ifdef _DEBUG
#define safeOutputDebug(x) printf( "%s", x );
#else
#define safeOutputDebug(x)
#endif

struct SplitscreenData {
    int  PLAYERCOUNT;
    int  globalSkill;
    int  globalEpisode;
    int  globalLevel;
    int  globalTimeLimit;
    int  globalFragLimit;
};

void      DL_InitNetworking( DoomInterface *pdi );
```

Outline

- Il primo programma C
- Il compilatore e il ciclo di sviluppo di un programma
- Sintassi C: Tipi, Keywords, Operatori, Espressioni, Identificatori, Enunciati
- Variabili e Assegnamento: stato della memoria
- SWAP 1

Outline

- Il primo programma C
- Il compilatore e il ciclo di sviluppo di un programma
- Sintassi C: Tipi, Keywords, Operatori, Espressioni, Identificatori, Enunciati
- Variabili e Assegnamento: stato della memoria
- SWAP 1

Primo programma in C

```
// fig02_01.c
// A first program in C.
#include <stdio.h>

// function main begins program execution
int main(void) {
    printf("Welcome to C!\n");
} // end function main
```

Primo programma in C

```
1. // fig02_01.c
```

```
2. // A first program in C.
```

```
3. #include <stdio.h>
```

```
4.
```

```
5. // function main begins program execution
```

```
6. int main(void) {
```

```
7.     printf("Welcome to C!\n");
```

```
8. } // end function main
```

Le linee (o sue parti) che iniziano con `'//'` sono commenti del programmatore e vengono ignorati dal compilatore

Le linee che iniziano con `'#'` sono istruzioni per il preprocessore: dichiarazione di variabili e funzioni per input/output da tastiera/ a schermo

La funzione main:

- è il punto di ingresso nel programma
- non riceve nessun parametro in input
- `'return 0'` implicito

La funzione **main**

- Un programma C è composto sempre da un certo numero di **funzioni**, fra cui almeno la funzione **main**, punto di ingresso al programma
- Per ora tutto il nostro codice sarà contenuto nella funzione **main**, quindi programma e funzione **main** saranno sinonimi
- La funzione **main** accetta parametri da command line, ma noi non li utilizzeremo per ora (richiedono uso degli array)
- La funzione **main** ritorna al SO un codice di successo/errore, ma noi non lo utilizzeremo (mancanza di return esplicito) poiché viene sempre inserito di *default*.

La funzione main: il corpo (*body*)

- Il codice della funzione main() deve essere racchiuso fra una coppia di parentesi graffe '{}'
- Il codice racchiuso fra parentesi graffe è detto *blocco di codice*
- I blocchi possono essere annidati

La funzione main: estetica

È buona regola (*best practice*):

- *commentare* ogni funzione: `'//'` (o `'/* ... */'`)
- *indentare* istruzioni di ogni blocco di codice, specialmente in caso di annidamenti. TAB o SPAZI (meglio)

Righe vuote e caratteri di spaziatura sono ignorati dal compilatore (*con qualche eccezione*), ma sono **UTILISSIMI** al programmatore

write your programs like if the person who is going to take over your code is a psychopatic that knows your address

La funzione `printf`

- Serve a stampare a terminale output alfanumerico
- Per ora solo *stringhe* di testo racchiuse fra doppi apici "

```
printf("Welcome to C!\n");
```

La sequenza `'\n'` è una *sequenza di escape*:

- `\n` -> a capo nel terminale
- `\\` -> il carattere backslash
- `\"` -> doppio apice
- `\a` -> alarm
- `\b` -> beep cicalino vecchi PC, ora suono di sistema

La direttiva include

- Le righe che iniziano per '#' sono direttive per il preprocessore, argomento che approfondirete in altri corsi
- La direttiva '#include' include un dato file in un altro
- I files '.h' di intestazioni (header) contengono la dichiarazione di funzioni, ad es. fornite in librerie di sistema (printf() in libc.so)
- L'header file stdio.h in dichiara funzioni come printf() e scanf()
- Tutti gli esempi seguenti includeranno l'header stdio.h, anche quando questo è omesso riportato per ragioni di spazio